

Homework 1

Alex Ojemann and Dehkontee Chea Cuppah

10.1:

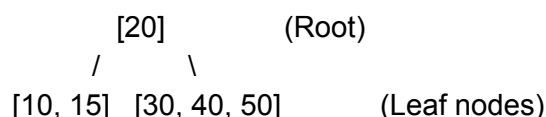
1. Inserting the data entry with key 9 into the tree would result in the same tree just with the entry with key 9 between those with keys 8 and 10 in the second to leftmost leaf node.
2. Inserting the data entry with key 3 would result in a tree like the one depicted but with the leftmost leaf node split into two leaf nodes with data entries 1 and 2 and 5 and 6 respectively, the left interior node being split into two interior nodes with data entries with keys 3 and 8 and 32 and 40 respectively, and the data entry with key 18 being added to the root node.
3. Deleting the data entry with key 8 and checking the left sibling for redistribution will result in a tree like the original one but with that data entry removed and the data entry with key 6 moved from the leftmost leaf node to the leftmost interior node.
4. Deleting the data entry with key 8 and checking the right sibling for redistribution will result in a tree like the original one but with that data entry removed and the second to leftmost and third to leftmost leaf nodes will be merged, with the merged node now containing the entries with keys 10, 18, and 27, and the data entry with key 18 removed from the leftmost interior node.

10.2:

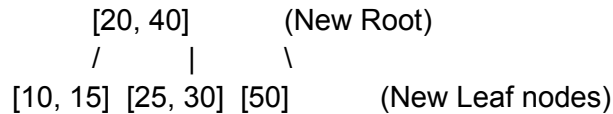
1. The nodes that must be accessed to get all records with search key >38 are I1, I2, I3, L2, L3, L4, L5, L6, L7, and L8
2. To insert a record with search key 109 into the tree, you would need to split L8 into two leaf nodes, L8 and L9. L8 will contain the records with search keys 98 and 99, L9 will contain the records with search keys 105 and 109, and the record with search key 100 will be moved up to I3. No additional splits will be required because I3 has space.
3. To delete the record with search key 81, that record is removed from L6 and the record with search key 79 is moved from L5 to L6 to avoid underflow in L6.
4. Inserting a record with search key 53 would require an increase in the height of the tree because it would require the splitting of L4 which is full, thus also I2, which is full, thus also I1, which is full and is the root node.

10.4:

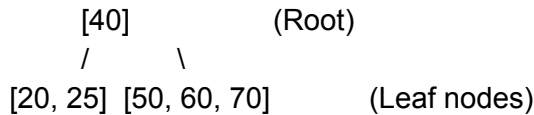
1. Before insertion:



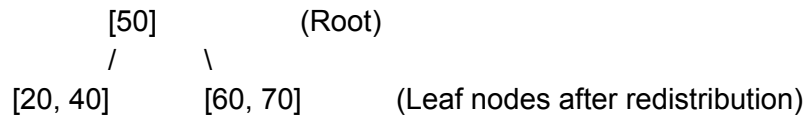
After insertion:



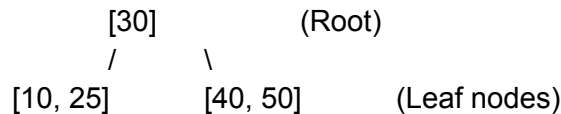
2. Before deletion:



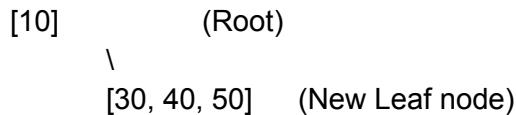
After Deletion:



3. Before deletion:



After Deletion:



11.3:

1. In extendible hashing, it is necessary to know the global depth because it is required to increase when any of the local depths exceed it to allocate enough space. It is necessary to know the local depth because when a bucket splits it increases the local depth by one, which allows the need for more storage to be fulfilled without requiring the global depth to increase which would require much more storage.
2. After an insertion causes the directory size to double, only two buckets will have one pointer because the number of pointers is $2^{(\text{global depth} - \text{local depth})}$ so only the buckets that caused the increase in global depth (by having the local depth that is equal to that new global depth) will have one pointer. If an entry is deleted from one of these buckets, assuming no other data had been added to it since the directory size doubled, the two buckets will be merged and the global depth will be decreased by one thus the directory size will be halved.

3. Yes, extendible hashing requires only one disk access to retrieve a record with a given key value because the hash function will find the correct bucket in main memory before accessing that bucket in disk storage.
4. If the distribution of data is non-uniform, then the global depth is much larger than the local depth in some instances and there will be storage space being wasted (the space utilization is poor).

12.2

- Consider a relation $R(a, b, e, d, e)$ containing 5,000,000 records, where each data page of the relation holds 10 records. R is organized as a sorted file with secondary indexes. Assume that $R.a$ is a candidate key for R , with values lying in the range 0 to 4,999,999, and that R is stored in $R.o$, order.

For each of the following relational algebra queries, state which of the following three approaches is most likely to be the cheapest:

- Access the sorted file for R directly.
- Use a (clustered) B+ tree index on attribute $R.o$.
- Use a linear hashed index on attribute $R.a$.

1. For this relational algebra query, using a clustered B+ tree index will be the cheapest due to its support for range queries.

2. For this relational algebra query, using the sorted file access will most likely be the cheapest due to the sorted nature of R .

3. For this relational algebra query, using a clustered B+ tree index will be the cheapest due to its support for range queries.

4. For this relational algebra query, using a clustered B+ tree index will be the cheapest due to its ability to skip to relevant records without having to scan everything.

12.4:

1. A. Yes, this matches the conditions, the primary conjunct is $Sailors.sid < 50000$.
B. Yes, this matches the conditions, the primary conjunct is $Sailors.sid = 50000$.
2. A. No, this doesn't match the conditions because hash indices can't do range searches.
B. Yes, this matches the conditions, the primary conjunct is $Sailors.sid = 50000$.

3. A. Yes, this matches the conditions, the primary conjuncts are Sailors.sid<50000 and Sailors.age=21.
 B. Yes, this matches the conditions, the primary conjuncts are Sailors.sid=50000 and Sailors.age>21.
 C. Yes, this matches the conditions, the primary conjunct is Sailors.sid<50000.
 D. No, this doesn't match the conditions because the leftmost key (Sailors.sid) isn't included
4. A. Yes, this matches the conditions, the primary conjuncts are Sailors.sid<50000 and Sailors.age=21.
 B. Yes, this matches the conditions, the primary conjuncts are Sailors.sid=50000 and Sailors.age>21. The hash index can work with the range search in this case because the results have already been filtered down to one record by the Sailors.sid=50000 selection which only selects one record because sid is the primary key which must be unique.
 C. Yes, this matches the conditions, the primary conjunct is Sailors.sid<50000.
 D. No, this doesn't match the conditions because the leftmost key (Sailors.sid) isn't included

16.2

1. **Give an example of another transaction T2 that, if run concurrently to transaction T without some form of concurrency control, could interfere with T1.**

If another transaction T2 tries to write to database X while transaction T1 is writing to database X concurrently without some form of concurrency control, it will lead to dirty reads but also write-write conflict.

2. **Explain how the use of Strict 2PL would prevent interference between the two transactions.**

For Strict 2PL to prevent interference between the two transactions when it is used, it will require transaction T2 to have an exclusive lock on X before it can write to it. Until t2 is committed, the lock will be held and this will block any operation from T1 on X till T2 is done.

3. **Strict 2PL is used in many database systems. Give two reasons for its popularity.**

Two reasons why Strict 2PL is used in many database systems are:

1. It is very simple and straightforward to implement. It helps with managing locks, ensuring data consistency and identifying deadlocks.

2. It prevents cascading abort and guarantees serializability.

16.4

We call a transaction that only reads database objects a read-only transaction, otherwise the transaction is called a read-write transaction. Give brief answers to the following questions:

1. What is lock thrashing and when does it occur?

Lock Thrashing is the point where transaction throughput decreases due to lots of conflicting lock requests from transactions and it occurs when the number of transactions running concurrently is too high.

2. What happens to the database system throughput if the number of read-write transactions is increased?

If the number of read-write transactions is increased, the database system throughput will decrease.

3. What happens to the database system throughput if the number of read-only transactions is increased?

If the number of read-only transactions is increased, the database system throughput will increase.

4. Describe three ways of tuning your system to increase transaction throughput.

Three ways to tune a system to increase transaction throughput are:

- Optimization of indexes
- Reduction in the time it takes for a transaction hold to lock
- Getting up to date hardware and software

16.8

Consider the following schema:

Suppliers(sid: integer, *sname*: string, *address*: string)

Parts(pid: integer, *pname*: string, *color*: string)

Catalog(sid: integer, *pid*: integer, *cost*: real)

The Catalog relation lists the prices charged for parts by Suppliers.

For each of the following transactions, state the SQL isolation level that you would use and explain why you chose it.

1. A transaction that adds a new part to a supplier's catalog.

For this transaction, the SQL isolation level that I will use is the READ COMMITTED isolation level. This transaction involves insertion and to avoid data inconsistencies, we must make sure the new part isn't read until it is fully committed, and this can be done with the READ COMMITTED isolation level. This level works best for this transaction as it will prevents partially inserted data from this transaction from being visible to other transactions.

2. A transaction that increases the price that a supplier charges for a part.

For this transaction, the SQL isolation level I will use is the REPEATABLE READ isolation level. The reason for choosing this level is because it will put mechanisms in place to ensure that if another running transaction tries to update the same price while this transaction is running, that transaction will be blocked until this transaction completes commitment. This level works best for data consistency and integrity.

3. A transaction that determines the total number of items for a given supplier.

For this transaction, the SQL isolation level that I will use is the READ COMMITTED isolation level. The reason for choosing this level is because it makes sure that this transaction only reads committed data for the aggregation.

4. A transaction that shows, for each part, the supplier that supplies the part at the lowest price.

For this transaction, the SQL isolation level I will use is the REPEATABLE READ isolation level. The reason for choosing this level is because this transaction requires reading multiple records and comparing their prices to identify the supplier with the lowest price of each part. The Repeatable Read isolation level then makes sure that once the transaction reads the existing prices of each part from a supplier, the prices will remain constant throughout the transaction.